ENVS: Environment-Native Verified Search for Long-Horizon GUI Agents

Yincheng Zhou[1,*] Athena Zhuoming Zhong[1,*] Shijie Zhang[2,*]
Kevin Zhang[2] Teresa Xiaotao Shang[1] Shanghang Zhang[2,†]

[1]*University of Pennsylvania* [2]*Peking University*

[*]Equal contribution. [†]Corresponding author.

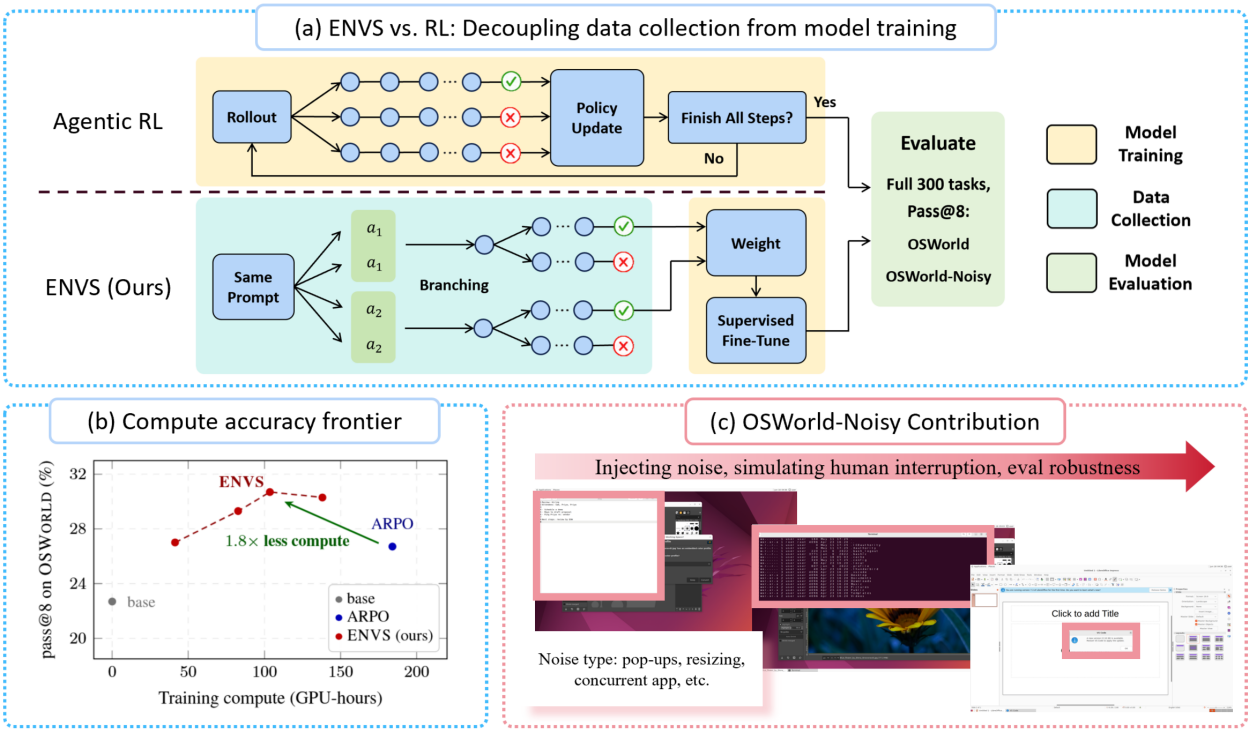

Figure 1: ENVS decouples data collection from model training **(a)**, reaching higher accuracy at lower compute than online RL **(b)**; OSWORLD-NOISY injects human-style interruptions to test robustness **(c)**.

Abstract

As multimodal agents move from interface understanding to real software control, successful trajectory discovery in live desktop environments becomes a key challenge. GUI tasks require long-horizon sequences of precise mouse and keyboard actions, while feedback is sparse, delayed, and costly to obtain through VM rollouts. We propose Environment-Native Verified Search (ENVS), a training-time search-and-filter pipeline that uses the environment to construct verified supervision before policy optimization: it branches over behaviorally distinct GUI actions in live OSWORLD VMs, verifies successful leaves, and trains from globally balanced step-level supervision. To evaluate robustness under realistic desktop interruptions, we also introduce OSWORLD-NOISY, a dynamic benchmark for recoverable desktop interruptions that preserves the original tasks while testing whether agents can refocus, dismiss, wait, or recover under live perturbations. On the 300-task OSWORLD pool, ENVS reaches 30.3 pass@8 on original evaluations and 29.0 on OSWORLD-NOISY, outperforming matched ARPO-style online RL while reducing compute from 184–192 to 138–153 GPU-hours; even with only 30% of its search data, ENVS reaches 27.0 pass@8, exceeding ARPO from the base model. Training from noisy environments also better preserves visual-reasoning abilities on auxiliary benchmarks, including OSWORLD-G REFUSAL (16.7 vs. 1.9) and BLINK FUNCTIONAL CORRESPONDENCE (26.2 vs. 23.1).

Code: `https://github.com/ArtysicistZ/ENVS`

1 Introduction

Multimodal foundation models are increasingly being used as agents that operate real software through screenshots, mouse actions, and keyboard actions [3, 13, 21, 27]. GUI agents are a demanding testbed for this progress because they require visual grounding, long-horizon planning, recovery from mistakes, and reliable execution in stateful environments.

Training such agents is difficult because success feedback is sparse, delayed, and expensive to obtain. In OSWORLD, each rollout executes inside a live desktop VM, and the environment usually provides only terminal task success or failure [40]. A failed trajectory often does not reveal which earlier action caused the failure. Thus, a central bottleneck is not only how to optimize a policy, but how to discover enough high-quality successful trajectories in the first place.

Recent work addresses sparse agent feedback through online RL and search. ARPO-style methods adapt GRPO with replay or rollout modifications for GUI and tool-use agents [20, 11, 26, 38], while tree-structured methods such as SEEA-R1 [36] integrate MCTS-style exploration with online policy optimization for long-horizon embodied agents. Search-based language-agent methods also explore multiple candidate reasoning or action paths before committing to a solution [42, 17, 25]. These methods motivate ENVS, but they still use environments mainly for online rollouts or inference-time search rather than for verified, balanced data construction before training. In online optimization, trajectory discovery remains coupled to policy updates: easier tasks can dominate the successful rollout distribution, and replay or clipping only improves local update stability without directly controlling the global allocation of gradient across tasks and trajectory lengths.

We propose ENVS (*Environment-Native Verified Search*). Rather than introducing a new reinforcement learning algorithm, ENVS is a search-based training-data construction framework that leverages environment-native verification signals to identify successful trajectories. Instead, it separates trajectory discovery from policy optimization. It searches in live OSWORLD VMs, branches over behaviorally distinct executable actions, verifies trajectory leaves with the OSWORLD oracle, and converts successful trajectories into globally balanced step-level supervised data. Unlike classical MCTS or Tree-GRPO, ENVS uses search only for training-time trajectory discovery, not online policy improvement. Because training happens after collection, ENVS can cap overrepresented easy tasks, upweight rare-success tasks, normalize long trajectories, and control how much gradient mass each part of the discovered dataset receives.

We also introduce OSWORLD-NOISY, a benchmark for evaluating GUI agents under recoverable live desktop interruptions. It preserves the original OSWORLD tasks and evaluators while injecting controlled pop-ups, focus changes, dialogs, overlays, and background activity that require agents to refocus, dismiss, wait, or recover before continuing. We evaluate ENVS on the 300-task OSWORLD evaluation pool, consisting of 86 trainable tasks and 214 held-out tasks, using the same pass@8 protocol as the ARPO-style baselines.

Our main findings are:

- **Clean performance:** ENVS reaches 30.3 pass@8 on OSWORLD, improving UI-TARS-1.5 from 22.7 and outperforming completed ARPO-style baselines at 26.7.
- **Noisy robustness:** On OSWORLD-NOISY, ENVS reaches 29.0 pass@8, compared with 20.3 for UI-TARS-1.5 and 21.7 for noisy ARPO from the base model.
- **Data quality:** A 30%-data ENVS variant reaches 27.0 pass@8, matching or exceeding ARPO from the base model.
- **Robustness trade-off:** Clean collection performs best on clean OSWORLD, while noisy collection slightly improves OSWORLD-NOISY and better preserves auxiliary capabilities, e.g., OSWORLD-G REFUSAL (16.7 vs. 1.9).

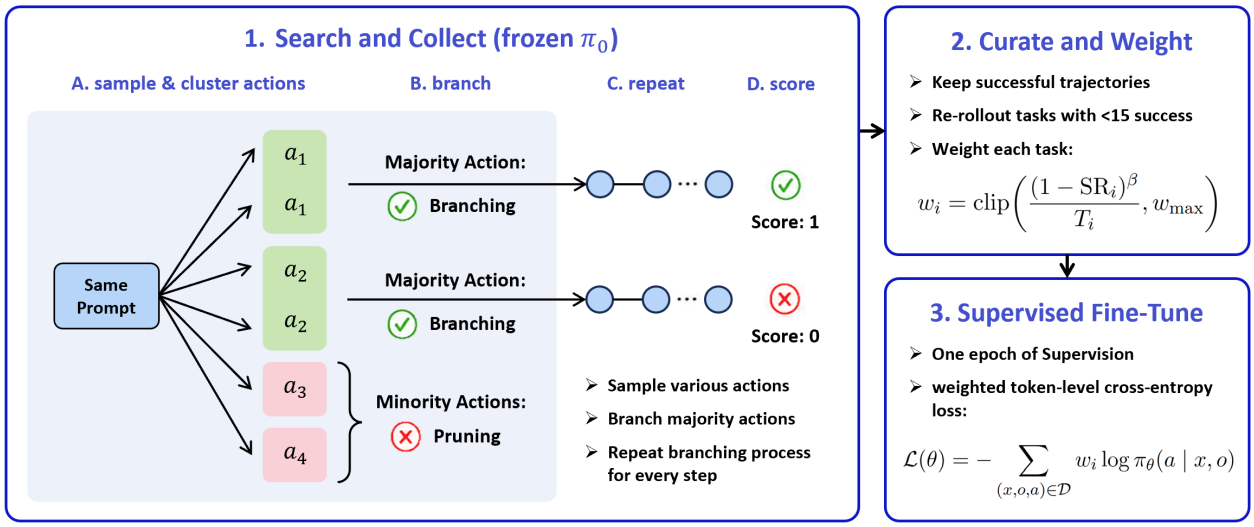

Figure 2: **ENVS pipeline overview**. ENVS uses environment-native tree search to collect verified successful trajectories from OSWORLD, curates them through filtering, weighting, and deduplication, and trains the agent with one-epoch SFT before evaluation on clean and noisy benchmarks.

2 Related Work

GUI agents and executable environments. Recent GUI-agent work studies how vision-language models map screenshots and instructions to executable actions across desktop, web, and mobile interfaces. OSWORLD provides real-desktop execution-based evaluation [40], while WebArena, VisualWebArena, AndroidWorld, and Mind2Web study related web and mobile settings [46, 16, 28, 9]. Model-side systems such as SeeClick, CogAgent, ScreenAgent, UI-TARS, OS-Atlas, and Aguvis improve GUI grounding and action prediction [3, 13, 21, 27, 39, 41]. Our work is complementary: we use executable GUI environments to construct verified training trajectories rather than only to evaluate or collect online rollouts.

RL, search, and verified supervision for agents. Reinforcement learning with verifiable rewards is widely used for language-model training [31, 32, 8, 43]. In GUI and agent settings, ARPO adapts GRPO-style optimization with replay [20], Agentic ARPO studies adaptive rollouts for tool-using agents [11], ToolRL and RAGEN train agents with environment or tool feedback [26, 38], and SEEA-R1 combines MCTS-style exploration with online policy optimization [36]. Tree search has also been used to turn sparse feedback into stronger policies, from MCTS/UCT and expert iteration to AlphaGo, AlphaZero, and MuZero [6, 15, 1, 33, 34, 30], and recent language-agent methods explore multiple reasoning or action paths before committing [42, 17, 25]. ENVS differs by using search only for training-time discovery of environment-verified demonstrations, followed by globally balanced supervised learning rather than online RL updates or deployment-time planning.

Data selection, imitation, verification, and robustness. ENVS is related to curriculum and data-selection methods such as CLPO, which emphasize informative examples during policy optimization [45], as well as imitation learning [24, 29] and verifier/process-supervision methods [5, 18, 12]. Its supervision, however, is generated by live GUI search and filtered by terminal execution success. OSWORLD-NOISY connects to work on curriculum learning, domain randomization, procedural generation, prioritized level replay, unsupervised environment design, and robust RL [2, 37, 4, 14, 10, 22, 23, 44, 35], but instantiates these ideas as recoverable desktop interruptions such as popups, overlays, focus shifts, and dialogs in executable GUI environments.

3 ENVS: Environment-Native Verified Search

ENVS is an SFT-based, training-time pipeline for OSWORLD GUI agents. It searches in live desktop VMs, verifies successful leaves, and converts them into balanced action-level supervision. This separation lets ENVS inspect the full discovered dataset before training, cap overrepresented tasks, normalize trajectory length, and control gradient allocation across tasks.

A *step* is one action index in a trajectory. A *node* is an environment branch: an executed action prefix and the desktop state reached by that prefix. An *alive node* is a branch still being expanded. When ENVS branches, the parent follows one action bucket, while child VMs replay the parent prefix and execute alternative behaviorally distinct actions.

3.1 Behavioral-disagreement-gated search

At each live node, ENVS samples K candidate next actions from a frozen policy and reduces each to a coarse behavior fingerprint: action type plus a discretization of its argument (quantized coordinates for clicks and drags, normalized text for typing, direction for scrolls, action type alone for zero-argument actions). Fingerprints are a cheap proxy, not a semantic equivalence — nearby clicks may hit different widgets, and distant clicks may hit the same large button. Candidates are bucketed by fingerprint, and the buckets are ranked by agreement: the number of sampled actions that fall in each bucket. The k highest-agreement buckets, where k is the per-step branch budget, are the *majority* actions and are explored; all lower-ranked buckets are *minority* actions and are pruned. If only one bucket is populated, the node continues along it; otherwise the single highest-agreement action is carried forward on the parent VM (so no VM is spent re-reaching the current state), and each remaining majority action spawns a child VM that replays the prefix and executes that divergent behavior. VM budget thus tracks behavioral divergence, not token noise.

3.2 VM-bounded search in live desktop environments

ENVS borrows the branching structure of tree search but not its standard machinery. Classical MCTS variants assume cheap, repeated simulations from abstract states and rely on UCB/PUCT selection with learned values [6, 15, 33, 34, 30]. In OSWorld, each branch is a live VM trajectory (real clicks, typing, application latency, focus state, brittle desktop transitions) and revisiting an abstract state is not free.

We therefore replace adaptive selection with explicit cost gates: a per-task branch budget, a per-step branch cap, late-step deferral, and a hard step cutoff. ENVS uses no learned value function, no UCB/PUCT, and no edge visit statistics. Its job is training-time trajectory discovery: finding executable paths that succeed under the OSWORLD evaluator and converting them to supervised data.

3.3 Environment verification and successful-leaf filtering

After search, the OSWorld task oracle assigns each leaf trajectory a binary reward $R(\tau) \in \{0, 1\}$. Failed trajectories are discarded; successful trajectories are decomposed into per-step supervised examples,

$$\mathcal{D} = \{(x, o_{\leq t}, a_t) : R(\tau) = 1,\ (o_{\leq t}, a_t) \in \tau\},$$

where x is the task instruction, $o_{\leq t}$ the observation history, and a_t the executed action. A single successful long-horizon trajectory thus yields many supervised targets, converting sparse terminal success into dense action-level supervision.

3.4 Global balancing of action-level supervision

Search produces an imbalanced dataset: easy tasks yield many successful leaves, long trajectories contribute many step examples, while hard tasks may yield few or none. Trained uniformly, the gradient is dominated by easy tasks and long trajectories.

ENVS rebalances after collection. Let $\mathcal{D}_i$ be the successful step examples for task i, $T_i = |\mathcal{D}_i|$, and SR_i its search success rate. The training objective is

$$\mathcal{L}(\theta) = \sum_i \sum_{(x, o_{\le t}, a_t) \in \mathcal{D}_i} w_i \left[-\log \pi_\theta(a_t \mid x, o_{\le t}, a_{<t}) \right],$$
$$w_i = \mathrm{clip}\left(\frac{(1 - \mathrm{SR}_i)^\beta}{T_i}, w_{\max} \right).$$

$(1 - \mathrm{SR}_i)^\beta$ shifts gradient mass to hard tasks; $1/T_i$ normalizes trajectory; $w_{\max}$ caps rare-success outliers. Smaller β ensures uniform weighting; larger β emphasizes hard tasks more aggressively.

This dataset-level rebalancing is the structural payoff of decoupling search from optimization: with the full verified dataset in hand before training, ENVS can flatten gradient allocation globally. Online RL gradients are determined by whatever rollouts the current policy produces, with no comparable handle on per-task gradient allocation.

4 OSWORLD-NOISY: Held-out Desktop Perturbations

Standard OSWORLD evaluation assumes a quiescent interaction loop in which the agent is the only active process on the desktop. In deployment this assumption rarely holds: focus changes, notifications, modal dialogs, and concurrent background activity routinely alter the visible state during task execution. OSWORLD-NOISY measures whether GUI agents can complete the original OSWORLD tasks under such interruptions. We model the perturbation source as a concurrent benign user who performs short, observable actions in non-target applications while the agent works on its task; the resulting friction is recoverable and is layered over the unmodified task and evaluator.

4.1 Perturbation templates

OSWORLD-NOISY draws from a library of 153 runtime noise generators spanning three categories: concurrent human-task sessions in non-target applications (e.g., note-taking in `gedit`, folder navigation in `nautilus`, or terminal activity); ambient interruptions (notifications, dialogs, browser prompts, toast overlays); and rare accidental target interference (partial occlusion, window shove, resize). Each generator returns a bash command that executes a short, observable interaction inside the OSWORLD container. A held-out subset of 24 generators is reserved exclusively for evaluation; the remaining 129 are available for noisy training and collection. The full catalog and held-out split are provided in Appendix B.1.

All perturbations satisfy three non-sabotage constraints: (i) noise does not deliver keystrokes or clicks to windows it has not itself opened and focused; (ii) noise does not touch files or paths referenced by the task evaluator; (iii) every perturbation is recoverable within a small number of standard agent actions. Under these constraints, OSWORLD-NOISY is a robustness setting rather than an impossibility benchmark: interruptions may distract or delay the agent but cannot render any task unsolvable.

4.2 Schedule sampler

A schedule specifies, for each rollout, the firing time of each perturbation, the bash command to execute, and metadata consumed by the curriculum. During training, perturbation count and difficulty scale with

the task's recent success rate: hard tasks receive little or no noise, while higher-success tasks receive more frequent or higher-cost events. This protects tasks the policy cannot yet solve and introduces interruption gradually as performance improves.

Evaluation uses a fixed protocol. Each task receives exactly one held-out perturbation with deterministic timing, and the schedule is shared across models via common random numbers. Clean and noisy evaluations are therefore directly comparable, and cross-model differences cannot be attributed to random variation in noise difficulty.

4.3 Train/evaluation split

The 24 held-out generators are reserved for evaluation; the underlying OSWORLD tasks are unchanged. OSWORLD-NOISY therefore measures recovery under unseen interruptions on the same task set, not performance on a different task set. This split follows the standard environment-generalization principle that robustness be evaluated on variations related to, but distinct from, those seen during training [4, 14, 10, 22]. In our setting, the variation is a recoverable GUI interruption rather than a procedurally generated level or simulated physics parameter.

4.4 Injection sites

The same library supports three uses. During ENVS collection, a fraction of search branches receive noise while the remainder serve as clean controls, producing a mix of task-solving and recovery trajectories. During on-policy RL rollout collection (e.g., ARPO), a single schedule is shared across the rollout group for a task, reducing environment-driven variance within the group. At evaluation, schedules are precomputed from the held-out templates and executed verbatim, ensuring that OSWORLD-NOISY is reproducible across models and runs.

5 Experiments and Results

5.1 Experimental protocol

We evaluate on a fixed 300-task OSWORLD pool consisting of 86 trainable tasks and 214 held-out tasks. The trainable subset follows the ARPO protocol: it contains tasks on which the base UI-TARS-1.5-7B policy occasionally succeeds, ensuring non-zero reward variation for group-relative online RL. The held-out tasks are never used for ENVS collection or ARPO training. We evaluate each method on clean OSWORLD and on OSWORLD-NOISY, which uses the same 300 tasks with one held-out recoverable perturbation per task.

All policies are initialized from UI-TARS-1.5-7B. We compare ENVS with the base policy and ARPO, a GRPO-based online-RL baseline with replay over successful trajectories. The primary metric is pass@8: a task is solved if at least one of eight rollouts sampled at temperature $T = 1.0$ succeeds within the 15-step horizon. All methods share the same evaluator, horizon, temperature, and sampling protocol. We report GPU hours because live VM execution dominates training cost; ARPO collects fresh on-policy rollouts during training, while ENVS collects verified trajectories once and reuses them for SFT variants.

5.2 Main results on OSWORLD and OSWORLD-NOISY

All methods start from UI-TARS-1.5-7B and are evaluated with the same 15-step pass@8 protocol on the 300-task pool. ENVS is the strongest method in both matched conditions (Table 1). On clean OSWORLD, ENVS reaches 30.3% overall pass@8, improving over the base model by 7.6 pp and ARPO-clean by 3.6 pp. The gain appears on both trained tasks (73.3% to 87.2%) and held-out tasks (2.3% to 7.5%).

On OSWORLD-NOISY, ENVS reaches 29.0%, compared with 20.3% for the base model and 21.7% for ARPO-noisy. The improvement is mainly concentrated on the trained subset: ENVS-noisy reaches 88.4%, while ARPO-noisy reaches 62.8% and falls below the noisy base model. On held-out tasks, ENVS-noisy and ARPO-noisy both reach 5.1%, so the noisy gain reflects stronger use of verified trainable-task supervision rather than broad held-out transfer.

Table 1: Main results on OSWORLD and OSWORLD-NOISY. We report pass@8 success rate (%) on the trained 86-task subset, the held-out subset, and overall, with absolute pp gain over the base policy in parentheses. The trained subset is selected following the ARPO protocol as the tasks on which the base policy occasionally succeeds. Each training method is evaluated only on its matched eval condition. Bold marks the column maximum within each block.

Model	Train data	Trained (86)	Held-out	Overall
OSWORLD *(clean)*				
UI-TARS-1.5-7B	—	73.3	2.3	22.7
+ ARPO	clean rollouts	81.4 (+8.1)	4.7 (+2.4)	26.7 (+4.0)
+ ENVS	clean trajectories	**87.2** (+**13.9**)	**7.5** (+**5.2**)	**30.3** (+**7.6**)
OSWORLD-NOISY				
UI-TARS-1.5-7B	—	66.3	1.9	20.3
+ ARPO	noisy rollouts	62.8 (−3.5)	5.1 (+3.2)	21.7 (+1.4)
+ ENVS	noisy trajectories	**88.4** (+**22.1**)	**5.1** (+**3.2**)	**29.0** (+**8.7**)

5.3 Compute efficiency

ENVS uses 138–153 total GPU-hours, compared with 184–192 for matched ARPO baselines (Table 2). The saving comes from decoupling collection from optimization: ENVS collects verified trajectories once and trains only successful trajectories with one-epoch SFT, whereas ARPO collects fresh on-policy rollouts inside each training run, updating policy with all trajectories.

Table 2: Per-method compute decomposition. ENVS rows split SFT GPU-hours from offline trajectory-collection GPU-hours; ARPO fuses rollout collection and policy updates on the training cluster (Collect column dashed). Bold marks the column minimum within each block.

Model	Method	Train GPU-h	Collect GPU-h	Total GPU-h
Clean-trained				
UI-TARS-1.5-7B	+ ARPO	184	—	184
UI-TARS-1.5-7B	+ ENVS	31	107	**138**
Noisy-trained				
UI-TARS-1.5-7B	+ ARPO	192	—	192
UI-TARS-1.5-7B	+ ENVS	32	121	**153**

Note. ARPO is an online policy-optimization method: rollout collection occurs inside the training loop on the same training GPUs, so collection cost is included in Train GPU-h and is not reported as a separate offline phase. ENVS decouples the phases, so trajectory collection and SFT training are reported separately.

5.4 Data efficiency

ENVS remains competitive with substantially less data. Using only 30% of collected trajectories, it reaches 27.0% pass@8, matching ARPO-clean at 26.7% (Figure 3). Performance increases to 30.7% at 75% data

and remains essentially saturated at 100% (30.3%). This suggests that verified search improves supervision quality, not only supervision volume.

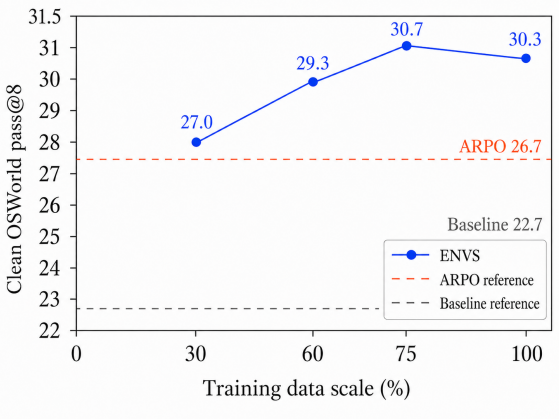

Figure 3: Clean OSWORLD pass@8 as a function of ENVS training data volume. The 30% subset already matches ARPO-clean, while gains saturate near the full dataset.

5.5 Clean versus noisy trajectory collection

Table 3 evaluates each trained policy on both clean and noisy test conditions. Under the matched ARPO configuration, noisy online rollouts degrade performance: ARPO-noisy reaches only 22.3% on clean OSWORLD and 21.7% on OSWORLD-NOISY, close to the base model and below ARPO-clean's noisy transfer score of 23.7%. Its trained-subset performance also drops below the noisy base model (66.3% to 62.8%; Table 1).

ENVS is more stable across conditions. ENVS-clean scores 30.3% on clean evaluation and 28.3% under noise, while ENVS-noisy scores 29.0% on both. Noisy collection therefore does not improve task completion over clean collection, but ENVS avoids the degradation observed in noisy online RL.

Table 3: Cross-evaluation of every trained policy on both the clean and noisy 300-task pools. Values are pass@8 in percent. Bold marks the column maximum. ARPO-noisy is the weakest learned policy and is matched by ARPO-clean's cross-condition transfer; ENVS-noisy reaches parity with ENVS-clean on both conditions.

Variant	Clean OSWorld	OSWORLD-NOISY
UI-TARS-1.5-7B	22.7	20.3
+ ENVS, clean	**30.3**	28.3
+ ENVS, noisy	29.0	**29.0**
+ ARPO, clean	26.7	23.7
+ ARPO, noisy	22.3	21.7

5.6 Auxiliary visual-reasoning capability retention

Table 4 evaluates whether GUI training preserves broader visual-reasoning abilities. Noisy ENVS matches or exceeds clean ENVS on all twelve benchmarks. The largest difference is OSWorld-G refusal: clean ENVS drops from 14.8% to 1.9%, while noisy ENVS reaches 16.7%. Noisy ENVS also improves over clean ENVS on BLINK Functional Correspondence (23.1% to 26.2%) and BLINK IQ Test (27.3% to 30.0%).

These results indicate a trade-off. Clean ENVS gives the best clean task-completion score, while noisy ENVS better preserves auxiliary visual-reasoning behavior. This suggests that recoverable interruptions

provide useful supervision for perception, waiting, and recovery, even when they do not improve clean task completion.

Table 4: Auxiliary visual-reasoning performance, grouped by capability channel. All scores are canonical benchmark accuracy (%) under VLMEvalKit deterministic decoding; no runtime perturbation is applied at evaluation. Bold marks the best of the three per row. Noisy ENVS matches or exceeds clean ENVS on every benchmark.

Benchmark	Base	Clean ENVS	Noisy ENVS
Refusal calibration			
OSWorld-G refusal	14.8	1.9	**16.7**
Vision-indispensable reasoning			
MathVerse-VO multi-choice	**78.0**	66.7	71.1
MMStar math	**52.8**	47.6	50.8
MMStar logical reasoning	**60.0**	56.0	**60.0**
MMStar science & technology	**38.4**	36.4	37.6
Visual-prior override			
HallusionBench VD-figure	68.8	67.5	**70.0**
HallusionBench VD-illusion	**63.2**	54.2	56.3
Cross-image differentiation			
MMVP per-pair	44.7	44.7	**46.7**
BLINK Semantic Correspondence	33.1	32.4	**33.8**
Abstract reasoning			
BLINK Functional Correspondence	23.1	23.1	**26.2**
BLINK IQ Test	27.3	27.3	**30.0**
Spatial reasoning			
BLINK Spatial Relation	87.4	87.4	**88.1**

6 Conclusion

Realistic GUI control brings embodied-AI challenges into software environments: agents must act through perception and action in stateful desktops where observations change, actions have delayed effects, and interruptions can occur during task execution. ENVS addresses this setting by turning sparse terminal feedback from live desktop environments into verified, balanced action-level supervision. It searches in OSWORLD VMs, branches over behaviorally distinct GUI actions, filters successful leaves with the environment oracle, and trains from globally balanced trajectories.

Across completed OSWORLD experiments, ENVS outperforms ARPO-style online RL on clean and noisy evaluation while using less compute. The reduced-data result shows that a small set of rich, diverse, environment-verified trajectories can reach online-RL-level performance, suggesting that carefully constructed SFT data can recover much of the benefit of RL without repeated on-policy VM rollouts. OSWORLD-NOISY further shows that dynamic perturbations are not only a robustness test, but also a source of supervision for refocusing, waiting, recovery, and careful perception.

More broadly, environment-native training can make agent training cheaper and more reusable by amortizing expensive environment interaction into verified trajectory datasets. At the same time, stronger GUI-control agents raise risks from unintended actions, unauthorized software control, and harmful automation. Our experiments are restricted to benchmark VMs with recoverable perturbations and task-specific evaluators; future work should pair environment-native training with permission boundaries, auditing, human oversight, and deployment-time safety constraints.

References

[1] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. *arXiv preprint arXiv:1705.08439*, 2017.

[2] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning*, pages 41–48, 2009.

[3] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents, 2024.

[4] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning*, 2020.

[5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

[6] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *International Conference on Computers and Games*, pages 72–83. Springer, 2006.

[7] Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models, 2025.

[8] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.

[9] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023.

[10] Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. In *Advances in Neural Information Processing Systems*, 2020.

[11] Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Agentic reinforced policy optimization, 2025.

[12] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (ReST) for language modeling, 2023.

[13] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. CogAgent: A visual language model for GUI agents, 2023.

[14] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, 2021.

[15] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.

[16] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Annual Meeting of the Association for Computational Linguistics*, 2024.

[17] Jing Yu Koh, Stephen McAleer, Daniel Fried, and Ruslan Salakhutdinov. Tree search for language model agents. In *International Conference on Learning Representations*, 2025. Also available as arXiv:2407.01476.

[18] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

[19] Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. ProRL: Prolonged reinforcement learning expands reasoning boundaries in large language models, 2025.

[20] Fanbin Lu, Zhisheng Zhong, Shu Liu, Chi-Wing Fu, and Jiaya Jia. ARPO: End-to-end policy optimization for GUI agents with experience replay, 2025.

[21] Runliang Niu, Jindong Li, Shiqi Wang, Yali Fu, Xiyu Hu, Xueyuan Leng, He Kong, Yi Chang, and Qi Wang. ScreenAgent: A vision language model-driven computer control agent, 2024.

[22] Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design. In *International Conference on Machine Learning*, 2022.

[23] Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust adversarial reinforcement learning. In *International Conference on Machine Learning*, 2017.

[24] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems*, 1989.

[25] Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. Agent Q: Advanced reasoning and learning for autonomous AI agents, 2024.

[26] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. ToolRL: Reward is all tool learning needs, 2025.

[27] Yujia Qin et al. UI-TARS: Pioneering automated GUI interaction with native agents, 2025.

[28] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents, 2024.

[29] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

[30] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

[31] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[32] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024.

[33] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

[34] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

[35] Reda Bahi Slaoui, William R. Clements, Jakob N. Foerster, and Sebastien Toth. Robust visual domain randomization for reinforcement learning. *arXiv preprint arXiv:1910.10537*, 2020.

[36] Wanxin Tian, Shijie Zhang, Kevin Zhang, Xiaowei Chi, Chunkai Fan, Junyu Lu, Yulin Luo, Qiang Zhou, Yiming Zhao, Ning Liu, Siyu Lin, Zhiyuan Qin, Xiaozhu Ju, Shanghang Zhang, and Jian Tang. Seea-r1: Tree-structured reinforcement fine-tuning for self-evolving embodied agents, 2025. URL https://arxiv.org/abs/2506.21669.

[37] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 23–30, 2017.

[38] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning, 2025.

[39] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A foundation action model for generalist GUI agents, 2024.

[40] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024.

[41] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous GUI interaction, 2024.

[42] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023.

[43] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale, 2025.

[44] Huan Zhang, Hongge Chen, Chaowei Xiao, Bo Li, Mingyan Liu, Duane Boning, and Cho-Jui Hsieh. Robust deep reinforcement learning against adversarial perturbations on state observations. In *Advances in Neural Information Processing Systems*, 2020.

[45] Shijie Zhang, Guohao Sun, Kevin Zhang, Xiang Guo, and Rujun Guo. Clpo: Curriculum learning meets policy optimization for llm reasoning, 2025. URL https://arxiv.org/abs/2509.25004.

[46] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, Tao Yu, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

A Preliminaries

A.1 GUI agents as partially observed control

An OSWORLD task specifies a natural-language instruction x and an initial desktop state. At each step t, the agent observes a context o_t consisting of the current screenshot and interaction history, then emits an executable GUI action a_t, such as clicking, typing, scrolling, pressing a hotkey, waiting, or terminating. Since the full desktop state is only partially observed through screenshots and actions can have delayed effects, we write the policy as history-conditioned:

$$\pi_\theta(a_t \mid x, o_{\leq t}, a_{<t}).$$

A trajectory is a sequence

$$\tau = (o_0, a_0, o_1, a_1, \ldots, o_H, a_H)$$

executed in a live virtual machine. Because actions modify desktop state, token-distinct actions can induce the same GUI behavior, while superficially similar actions can lead to different future states.

A.2 Sparse terminal verification in GUI environments

OSWORLD provides task-specific evaluators that judge whether a completed trajectory satisfies the instruction. For ENVS, we use this evaluator as a binary terminal verifier,

$$R_{\text{env}}(\tau) \in \{0, 1\}.$$

This signal is sparse: most intermediate states do not carry direct supervision, and a terminal failure does not identify which earlier action caused the failure. Online RL methods use related terminal reward signals to update the current policy from its own rollouts. Depending on the method, these rewards may include shaping terms or penalties for invalid outputs rather than being strictly binary, but they remain sparse because they are assigned primarily at the trajectory or final-output level [11, 20]. ENVS uses the environment signal differently: it treats $R_{\text{env}}(\tau)$ as a verifier for search-generated trajectories, filters successful leaves, and then trains from the resulting verified supervision.

B Additional implementation details

B.1 Noise template catalog

OSWORLD-NOISY contains 153 runtime noise generators. A held-out subset of 24 generators is used only for evaluation. Table 5 gives the expanded taxonomy used by the implementation. Counts refer to generator entries; a single generator can produce many concrete events through internal randomization.

B.2 Noise scheduling

During training, the number and difficulty of perturbations depend on task success. Hard tasks receive little or no noise; easier tasks receive more frequent and higher-cost events. At evaluation, each task receives exactly one held-out perturbation event. The evaluation schedule is deterministic, so every model sees the same interruption for the same task.

Table 5: Expanded taxonomy of OSWORLD-NOISY perturbations. Counts are generator entries; one generator can produce many concrete events through internal randomization.

Tier (count)	Cost	Representative perturbations	Recovery behavior
T1: concurrent human-task session (45)	0–1	`gedit` notes and drafts; `gnome-terminal` commands; `nautilus` folder browsing; settings panels; window switching and dragging.	Refocus the target application. Noise does not deliver input to the target window.
T2: ambient background and interruption (99)	0–3	Notifications, update/backup dialogs, fake downloads, browser prompts, permission dialogs, OS toasts, banners, modal overlays, compositional notification/modal events.	Dismiss, press `Esc`, ignore, wait, or dismiss components in sequence.
T3: accidental target interference (9)	1–2	Target-window shove, shrink, partial overlap, cover-from-above, transient network/proxy prompts.	Refocus, move, resize, or wait for auto-recovery.

Fire count.

$$N(\hat{s}) = \begin{cases} 0, & \hat{s} < 0.10, \\ 1, & 0.10 \le \hat{s} < 0.25, \\ \mathrm{Unif}\{1,2,3\}, & 0.25 \le \hat{s} < 0.50, \\ \mathrm{Unif}\{3,4,5\}, & \hat{s} \ge 0.50. \end{cases}$$

At evaluation, $N \equiv 1$.

Step placement. Fire steps are bucket-spaced across the rollout horizon. Setup steps are protected, and each scheduled event leaves enough remaining steps for recovery and task completion.

Determinism. Training and collection schedules are seeded by task and rollout identifiers. Evaluation schedules are precomputed from the held-out generator set and executed verbatim across all models.

C Evaluation pool construction

We build the 300-task evaluation pool by applying two filters to the public OSWORLD test suite of 368 tasks across 10 application categories.

Filter 1: soft-reset stability (-13). A task is included only if its evaluator returns a deterministic outcome when the desktop is reset via OSWorld's soft-reset path (revert VM to a named snapshot rather than a full re-provision). Tasks whose evaluators fail intermittently under soft reset — typically because they depend on filesystem state that the snapshot does not restore — are removed. This filter excludes 13 tasks; the surviving set is the `test_all_softreset.json` list of 355 tasks.

Filter 2: proxy-free reachability (-55). A task is included only if every URL its trajectory reaches can be served without an authenticated outbound HTTP proxy. Browser tasks that depend on gated services (logged-in social sites, region-locked endpoints, accounts we cannot anonymously instantiate) are removed. This filter excludes a further 55 tasks, leaving the 300-task evaluation pool used in this paper.

The two filters together exclude 68 of 368 tasks. The exclusions are concentrated in two app categories whose tasks are predominantly browser- or web-mediated (Table 6); GIMP, LibreOffice Calc, LibreOffice Writer, and VLC are unaffected.

Table 6: Per-app composition of the evaluation pool. The 300-task pool preserves all 10 OSWORLD application categories. Exclusions are concentrated in `chrome` and `multi_apps`; both are dominated by browser- and network-mediated tasks affected by Filter 2.

Application	Full (368)	Eval pool (300)	Excluded
`chrome`	46	16	30
`gimp`	26	26	0
`libreoffice_calc`	47	47	0
`libreoffice_impress`	47	46	1
`libreoffice_writer`	23	23	0
`multi_apps`	101	72	29
`os`	24	19	5
`thunderbird`	15	14	1
`vlc`	17	17	0
`vs_code`	22	20	2
Total	368	300	68

Train / eval relationship. The 86-task trainable subset used for ENVS collection and ARPO training is a strict subset of the 300-task evaluation pool ($86 \subset 300$). Pass@8 numbers reported in Section 5 therefore include both trained-on tasks and 214 held-out tasks; the train / held-out split is broken out in Table 1.

Exact list. The 300 retained task IDs and their per-app grouping are shipped verbatim in the supplementary archive as `evaluation_examples/test_all_300tasks_noproxy_softreset_clean.json`. The 355-task soft-reset-filtered intermediate set is provided as `evaluation_examples/test_all_softreset.json` for readers who wish to reproduce the two filter stages independently.

D Structural Advantages of ENVS

The empirical results in Tables 1–3 show that ENVS outperforms ARPO on long-horizon, sparse-reward GUI control under both clean and noisy evaluation. This appendix identifies four structural properties of online policy gradient that account for the gap and shows that each is removed by construction in ENVS: a binomial degeneracy in per-task gradient allocation, monotone contraction of action entropy under positive-covariance updates, non-convexity of the return objective combined with unbounded equilibrium drift, and the inability to reuse rollouts across training variants. Notation follows the main body throughout: π_θ for the trained policy, π_0 for UI-TARS-1.5-7B, $\mathcal{D}_i$ for the supervised step set on task i with $T_i = |\mathcal{D}_i|$, and SR_i for the empirical search success rate on task i. We additionally write $q(\cdot \mid s)$ for the empirical action distribution at state s implied by $\bigcup_i \mathcal{D}_i$, and $H[\pi(\cdot \mid s)]$ for the Shannon entropy of π at state s.

D.1 Gradient Degeneracy

Under independent Bernoulli rollouts with per-trajectory success probability p_i on task i, the number of successes k in a group of size G follows $\mathrm{Bin}(G, p_i)$. The probability that the group contains both at least one success and at least one failure is therefore

$$\Pr[\text{non-degenerate}] \;=\; 1 - \Pr[k = 0] - \Pr[k = G] \;=\; 1 - (1 - p_i)^G - p_i^G.$$

The (uncorrected) sample variance of G Bernoulli outcomes with k successes is

$$\hat{\sigma}_r^2 \;=\; \frac{1}{G}\sum_{j=1}^{G}(r_j-\bar{r})^2 \;=\; \bar{r}\,(1-\bar{r}) \;=\; \frac{k}{G}\left(1-\frac{k}{G}\right),$$

which equals zero when $k \in \{0, G\}$. The implementation guards the resulting division in the standardised advantage by an ε, sending all advantages on a degenerate group to approximately zero; direct calculation on a non-degenerate group gives $\sum_{j=1}^{G}\hat{A}_j^2 = G$. Per-task expected gradient mass therefore factorises as the non-degenerate probability above multiplied by the conditional variance $\mathbb{E}[\hat{\sigma}_r^2 \mid 0 < k < G]$, both of which are maximised at $p_i = 1/2$ and decay symmetrically. At the configuration used in our ARPO runs ($G = 8$), the combined factor at $p_i = 0.05$ is approximately a sixth of its value at $p_i = 0.5$. Tasks the current policy almost never solves and tasks it almost always solves are systematically attenuated regardless of G or replay strategy, since replay over successful trajectories cannot manufacture successes for tasks that have none.

ENVS computes per-task weights from the full collected dataset rather than from a current rollout group. Following the loss in §4.4, the step weight is

$$w_i \;=\; \mathrm{clip}\left(\frac{(1-\mathrm{SR}_i)^{\beta}}{T_i},\; w_{\max}\right),$$

and summing over the T_i step samples of task i gives the per-task aggregate gradient weight

$$\sum_{(s,a)\in\mathcal{D}_i} w_i \;=\; T_i \cdot w_i \;=\; (1-\mathrm{SR}_i)^{\beta} \quad \text{(pre-clip)},$$

which is monotonically increasing in task difficulty $1-\mathrm{SR}_i$ and independent of trajectory length, with the exponent β controlling the strength of the correction. Allocation across tasks is specified in closed form before any gradient step is taken, and the binomial degeneracy of GRPO does not arise. The trained-set greedy regression of ARPO-noisy, $54/86$ against $57/86$ for the base policy in Table 1, is the behavioural signature of unbalanced allocation that this construction is designed to avoid.

D.2 Entropy Collapse

For a softmax policy π_n updated by policy gradient with step size η, Cui et al. [7] show that

$$\Delta H_n \;=\; -\eta \cdot \mathbb{E}_{s\sim d^{\pi_n}}\left[\mathrm{Cov}_{a\sim\pi_n(\cdot|s)}\big(\log \pi_n(a \mid s),\, A^{\pi_n}(s,a)\big)\right] + O(\eta^2),$$

where we use n for the update index to avoid collision with the trajectory step t in §3.1. They report empirically that this covariance is positive throughout training across PPO, GRPO, RLOO, and Reinforce++, so $\Delta H_n \le 0$ at every step under that regime. Contractions compound because update $n+1$ samples under the already-narrower π_n, and online policy gradient cannot increase action entropy without an explicit entropy bonus.

The supervised loss has the opposite floor. Per state s, the supervised loss is the cross-entropy of $\pi_\theta(\cdot \mid s)$ relative to the empirical $q(\cdot \mid s)$,

$$\mathcal{L}_s(\theta) \;=\; -\sum_a q(a \mid s)\, \log \pi_\theta(a \mid s).$$

Adding and subtracting $\sum_a q(a \mid s)\, \log q(a \mid s)$ on the right gives

$$\mathcal{L}_s(\theta) \;=\; -\sum_a q(a \mid s)\, \log q(a \mid s) + \sum_a q(a \mid s)\, \log \frac{q(a \mid s)}{\pi_\theta(a \mid s)},$$

in which the first term is the Shannon entropy $H[q(\cdot \mid s)]$ and the second is the KL divergence from q to π_θ. By Gibbs' inequality the KL term is non-negative, with equality iff $\pi_\theta = q$ on $\mathrm{supp}(q)$, so

$$\mathcal{L}_s(\theta) \;=\; H[q(\cdot \mid s)] + \mathrm{KL}\big(q(\cdot \mid s) \,\big\|\, \pi_\theta(\cdot \mid s)\big) \;\geq\; H[q(\cdot \mid s)],$$

attained at the supervised optimum $\pi^* = q$. The converged per-state entropy therefore equals $H[q]$, which exceeds $H[\pi_0]$ whenever the empirical distribution q is broader than π_0 at state s. Because branching retains the $k = 3$ highest-agreement action clusters (one on the parent VM and up to two on child VMs) drawn from $K = 32$ candidates, $|\mathrm{supp}(q)| \leq 3$ at any branching state; when the surviving branches contribute roughly comparable mass to q (a property of our search-leaf statistics rather than a design constraint), $H[q] \approx \log_2 |\mathrm{supp}(q)|$, which exceeds $H[\pi_0]$ on every state where π_0 is concentrated below the corresponding entropy ceiling.

Whether this condition is met in practice across our held-out states is an empirical question. Table 7 reports per-state action entropy on 54 held-out states, and the measured ordering is

$$H[\pi_{\text{ENVS}}] \;>\; H[\pi_0] \;>\; H[\pi_{\text{ARPO}}] \quad (1.640 > 1.282 > 0.887 \text{ bits}),$$

matching both halves of the structural argument: ENVS sits above the base policy, while ARPO sits below it.

Table 7: Held-out per-state action entropy (H_{action}, in bits) and effective number of action categories ($2^{H_{\text{action}}}$) on 54 states (12 tasks $\times$ 5 early steps each, stratified by base-policy success-count tier). For each policy, $K = 8$ samples at temperature $T = 1$ are drawn at every state, and Shannon entropy is computed over the joint action-type and quantised spatial-coordinate fingerprint. Pass@8 reproduced from Table 1.

Policy	H_{action}	$2^{H_{\text{action}}}$	pass@8
UI-TARS-1.5-7B	1.282	2.43	22.7
ARPO-clean	0.887	1.85	26.7
ENVS-clean	1.640	3.12	30.3
ENVS-noisy	1.729	3.31	29.0

D.3 Instability: Non-convexity and Drift

The reinforcement-learning return

$$J(\pi) \;=\; \sum_{\tau} \rho_0(s_0) \prod_{t=0}^{H-1} \mathcal{T}(s_{t+1} \mid s_t, a_t)\, \pi(a_t \mid s_t)\, R_{\text{env}}(\tau),$$

contains the per-state policy probabilities $\pi(\cdot \mid s)$ as factors that appear once per visit to state s along τ, so J is multilinear of degree at most H in the joint per-state simplex. Even the bilinear case $f(x, y) = xy$ has indefinite Hessian, so J is non-convex in general. The policy-gradient theorem therefore guarantees only stationary-point convergence, and PPO/GRPO additionally apply a non-differentiable importance-ratio clipping at $[1-\epsilon, 1+\epsilon]$ that prevents direct convex analysis. Equilibrium drift $\mathrm{KL}(\pi_n \,\|\, \pi_0)$ admits no a priori bound: it is controlled in practice only by an explicit KL coefficient in the objective [31] or by periodic resets to a reference policy [19], neither of which is part of the ARPO configuration used in our experiments. Yu et al. [43] report that production-scale GRPO requires four interventions (Dynamic Sampling, Clip-Higher, token-level loss, overlong reward shaping) to remain stable, each addressing an instability that the underlying objective does not itself control.

The supervised loss is by contrast convex in logit space, with an explicit a priori bound on equilibrium drift from π_0. Writing $\pi_\theta(a \mid s) = \mathrm{softmax}(z_\theta(s))_a$, the first derivative of the log-sum-exp $\mathrm{LSE}(z) = \log \sum_b \exp(z_b)$ is $\partial \mathrm{LSE}/\partial z_i = e^{z_i} / \sum_b e^{z_b} = \pi_i$, and differentiating once more by the quotient rule yields

$$H_{ij} \;=\; \frac{\partial^2 \mathrm{LSE}(z)}{\partial z_i \, \partial z_j} \;=\; \pi_i \, \delta_{ij} - \pi_i \pi_j.$$

For any $v \in \mathbb{R}^{|\mathcal{A}|}$,

$$v^\top H v \;=\; \sum_i \pi_i v_i^2 - \Big(\sum_i \pi_i v_i \Big)^2 \;=\; \mathbb{E}_\pi[v^2] - (\mathbb{E}_\pi[v])^2 \;=\; \mathrm{Var}_\pi(v) \;\geq\; 0,$$

so H is positive semi-definite and the per-state cross-entropy $-z_a + \mathrm{LSE}(z)$ is convex in z (the linear term contributes zero to the Hessian). The loss is therefore bounded below by $H[q]$ on the entire logit space, attained at $\pi = q$.

The KL bound at the supervised optimum follows from the same algebra. For each $a \in \mathrm{supp}(q)$, $q(a)/\pi_0(a) \leq \max_b q(b) \,/\, \pi_0^{\min}(s)$, where $\pi_0^{\min}(s) = \min_{b \in \mathrm{supp}(q)} \pi_0(b \mid s)$, so

$$\mathrm{KL}\big(q(\cdot \mid s) \,\big\|\, \pi_0(\cdot \mid s)\big) \;=\; \sum_a q(a) \, \log_2 \frac{q(a)}{\pi_0(a)} \;\leq\; \log_2 \frac{\max_a q(a)}{\pi_0^{\min}(s)} \cdot \sum_a q(a) \;=\; \log_2 \frac{\max_a q(a)}{\pi_0^{\min}(s)}.$$

Any cluster retained by ENVS appeared in at least one of $K = 32$ samples drawn from π_0, so its empirical base frequency is at least $1/K$. Treating this empirical floor as a working lower bound on $\pi_0^{\min}(s)$ (a heuristic, since a cluster with true probability below $1/K$ can survive the retention rule by chance), the per-state KL at branching states is bounded by approximately $\log_2 K = 5$ bits; on non-branching states q concentrates near the modal action of π_0 and the per-state KL is small. Online RL admits no analogous a priori bound on $\mathrm{KL}(\pi_n \,\|\, \pi_0)$, even at convergence; the trained-set regression $54/86 < 57/86$ in Table 1 is the training-time signature of this absence.

D.4 Rollout Coupling

Online policy gradient updates the current policy from rollouts collected under that same policy. Reusing rollouts after the policy has changed requires importance reweighting, which is not part of the GRPO recipe implemented by ARPO in our experiments. Each ablation therefore requires a fresh on-policy collection at the full per-run cost, since past trajectories were generated under a now-stale policy.

ENVS trajectories are collected under the frozen base policy π_0 together with branching, so the dataset is policy-independent and not specific to any downstream supervised variant. A single search collection underlies every supervised configuration in this paper at the cost of an additional supervised pass per variant. The headline GPU-hour numbers and the data-volume sweep enabled by this amortisation appear in Table 2.

E Limitations

Our experiments focus on OSWORLD and one base-model family, UI-TARS-1.5-7B. The quantitative results should therefore be read as claims about the studied OSWORLD regime, not as universal claims about all GUI agents. Additional experiments on browser, mobile, and other desktop environments are needed to measure transfer.

ENVS also requires infrastructure that is available in OSWORLD but not guaranteed in less controlled settings: parallel VM execution, reliable reset, and a terminal oracle for task success. Its behavior fingerprints are practical approximations rather than semantic equivalence classes; two actions with different

fingerprints can still be functionally equivalent, and two actions with similar fingerprints can diverge after execution.

OSWORLD-NOISY is synthetic and recoverable by design. Its perturbations approximate common desktop interruptions, but they do not cover the full range of real user behavior, system failures, or adversarial interference. Our results also show that noisy collection does not automatically improve OSWORLD task completion; perturbations need filtering and balancing to avoid diluting task-solving supervision.

Finally, our ARPO comparisons use the completed configurations in our study, and we do not claim to exhaust all online RL variants or hyperparameter choices. The experiments are expensive, and we do not yet report repeated training seeds for every method. We therefore emphasize matched protocols, compute accounting, and scoped conclusions rather than statistical dominance across all possible runs.